Realización de pruebas.

Caso práctico

Ahora que ya sabe cómo empaquetar y distribuir aplicaciones, **Ana** se da cuenta de que antes de que la aplicación llegue a su destino final, a manos del cliente, tiene que probarla para asegurarse de que tenga los mínimos errores posibles.

Ada tiene muy en cuenta este aspecto. Efectivamente, desde que fundara BK Programación, la empresa siempre ha sido reconocida por la calidad de su software. Esto ha hecho contar con una amplia cartera de clientes que siempre ha confiado en los proyectos realizados por la empresa.

Las directrices de **Ada** son muy precisas: en BK Programación se hace software de calidad, y esa calidad sólo se consigue destinando recursos económicos y temporales a la realización de un completo proceso de pruebas.



pixabay.com. *Claves de un proyecto* (Licencia Pixabay)

Todos conocen el tipo de pruebas que hay que realizar a las aplicaciones antes de presentarlas a los clientes. Por eso, **Ana**, que no quiere quedarse atrás, tiene que ponerse al día para participar en la estrategia de pruebas de las aplicaciones en las que está trabajando BK.

"No podemos ni pretendemos construir software 100 % libre de errores", comenta **Ada**, "pero con una ESTRATEGIA adecuada de pruebas, conseguiremos minimizarlos. Nuestro software ganará calidad y los costes del proyecto disminuirán. Eso es lo fundamental".



Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

Aviso Legal

1.- Pruebas de software.

Caso práctico

María va a ser la encargada de supervisar todas las pruebas a las que el software va a ser sometido durante el proceso. Les ha comentado a Ana y Antonio que pueden acompañarlos para que aprendan qué tipo de pruebas hay que realizar a la aplicación y la importancia de seguir un protocolo que las estandarice.

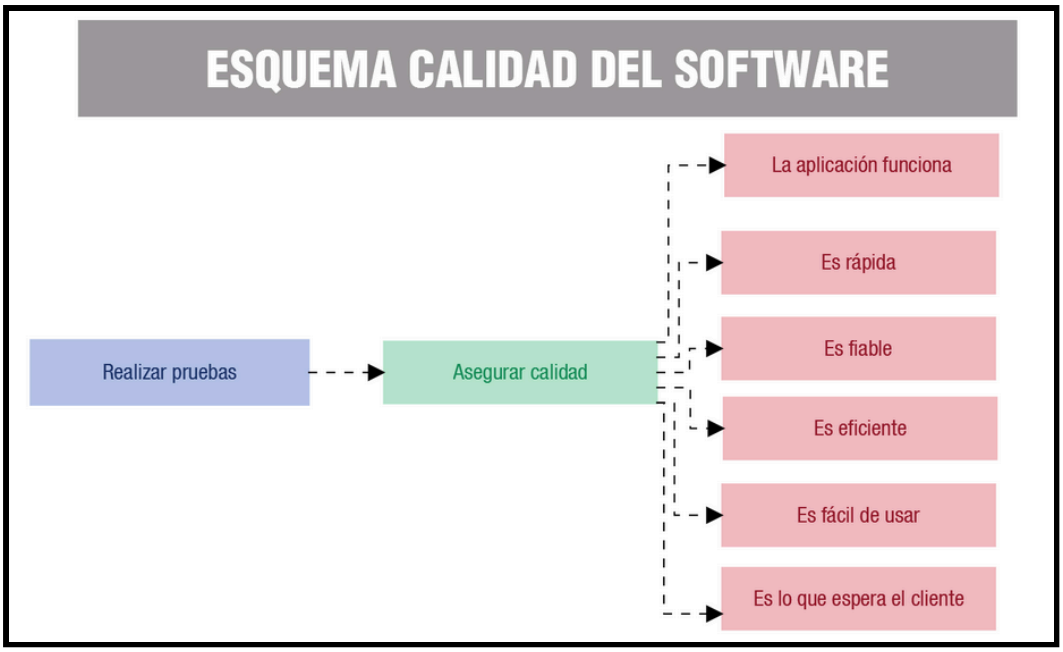


pixabay.com. *Pruebas* (Licencia Pixabay)

En múltiples ocasiones, a lo largo de los distintos módulos que componen el ciclo, se comenta que la **calidad del software** que produzcamos es un requisito indispensable.

La calidad no se da a la aplicación una vez está construida, sino que es una forma de pensar y actuar del programador a lo largo de todas las fases del proyecto.

Calidad no es sólo que el software funcione bien y que vaya rápido, sino que además implica que satisfaga las expectativas de los clientes, usabilidad, confiabilidad, coste, eficiencia y cumplimiento de los estándares.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Para asegurar la calidad, debemos hacer uso de las **pruebas de software**.

Asegurar los requisitos de calidad de software lleva un coste y trabajo asociado.

La realización de pruebas de software consume tiempo y recursos, pero es fundamental para que los fallos de la aplicación sean mínimos.

Serán los usuarios, clientes de nuestro proyecto, quienes impongan los requisitos y los que determinen si se ha conseguido o no nuestro objetivo de proporcionar un software fiable, correcto y de calidad.

Es imposible que los desarrolladores de software podamos saber de antemano, de forma total y absoluta, las necesidades que los clientes pueden tener en cada momento y puede ocurrir que a veces malinterpretemos ciertos requisitos. Es por ello que, como fase final en la estrategia de pruebas, el software termine siendo utilizado por los clientes durante un período de prueba, para que sean ellos quienes concluyan si nuestra aplicación es la que ellos demandaron en un principio o si es necesario aplicar alguna modificación.

La ausencia o la mala gestión de una estrategia de pruebas en un proyecto software pueden suponer:

- Falta de satisfacción de los clientes.
- Fallos en la explotación por defectos no detectados y eliminados a tiempo.
- Altos costes finales, por el tiempo empleado en corregir defectos.

Para saber más

En el siguiente enlace encontrarás un documento muy interesante sobre la importancia de realizar software de calidad en nuestra empresa, así como otra información que te será de interés en la presente unidad.

Importancia de las pruebas al software.

1.1.- Objetivos.

Caso práctico

"No hace falta ser un experto para saber que tenemos que encontrar todos los defectos, "comenta **Antonio**". "Te equivocas", le corrige **María**. Por su propia naturaleza, el software siempre tendrá defectos. Nuestro objetivo es minimizarlos.



pixabay.com. *Riesgos* (Licencia Pixabay)

El objetivo de las pruebas al software es encontrar la mayor cantidad posible de errores.

Todas las aplicaciones que se construyen tienen defectos. El origen de los mismos puede ser de distinta naturaleza, pero tienen un denominador común: hay que detectarlos y eliminarlos.

En todo proyecto software se debe perseguir que los errores sean mínimos (exigir la no existencia de errores en un proyecto software es una utopía) y que el proceso de encontrarlos y corregirlos lleve asociado el mínimo coste posible.

Por ello, es imprescindible desarrollar la mejor estrategia de pruebas en nuestro proyecto.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Descubrir un error se considera el éxito de una prueba.

No sólo basta con "ejecutar código", sino que hay que hacerlo mediante requisitos, especificaciones funcionales, diseños, etc. Es decir, hay que verificar y validar el software.

El objetivo inmediato es evitar que los defectos se propaguen de unas fases a otras hasta llegar a los sistemas de producción.

En definitiva, buscamos **reducir** los riesgos por la aparición de defectos en los procesos de implantación y explotación de las aplicaciones. Todo ello mediante la detección temprana de problemas y errores. Con todo esto se pretende desarrollar un nivel de confianza sobre la disponibilidad (en tiempo) y la fiabilidad (a lo largo del tiempo) de los productos desarrollados, acorde a las expectativas del cliente.

Las pruebas deben entenderse como:

- Verificación: El producto que se está construyendo funciona correctamente; es decir, es capaz de realizar la tarea para la cual ha sido diseñado.
- Validación: El producto terminado, además de ser correcto, es conforme con lo que el cliente había esperado.

Autoevaluación

Pregunta

Piensa en una aplicación que se acaba de construir. Entre otras muchas cosas, tiene un buscador que conecta a una base de datos. Cuando localizamos a una persona, se muestran en una pantalla sus datos. El cliente había pedido que también se mostrara una foto de esa persona, la cual no aparece. ¿Qué podemos decir de esa aplicación?

Respuestas

La aplicación ha sido verificada y validada.

La aplicación ha sido verificada, pero no validada.

La aplicación no ha sido ni verificada ni validada.

Retroalimentación

Incorrecto. No está validada puesto que no cumple con los requisitos del cliente.

Así es. Vas captando la idea.

Incorrecto. La aplicación funciona y hace lo que se le pide, por tanto, está verificada.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

1.2.- Importancia.

Caso práctico

"¡Qué más dará encontrar los defectos antes o después!" dice **Antonio**. "Cuanto antes se encuentren, menos costes en corregirlos". le reprocha **María**.



pixabay.com. *Ahorro* (Licencia Pixabay)

El único recurso de que disponemos los desarrolladores de software para determinar y acreditar el nivel de calidad de nuestros productos software es el proceso de pruebas.

En este proceso se ejecutan pruebas dirigidas a los componentes software individualmente y al software en su totalidad, con el objetivo de medir el grado en que el software cumple con los requisitos impuestos.

Cuanto más tarde es encontrado un defecto más cuesta (en tiempo, recursos y dinero) su corrección.

Por tanto, hemos de intentar identificar y resolver los defectos tan pronto como son originados con la mejora de los procesos abiertos.

Como resultado, entregaremos aplicaciones de alta calidad en tiempo cobertura y costes, en línea con lo establecido.

Por tanto, la estrategia de pruebas contribuye al ahorro de coste del proyecto y a crear un software de mayor calidad. En concreto, permite:

Mejorar la satisfacción de la clientela.

Incrementar la productividad.

Disminuir los riesgos del negocio.

Facilitar el crecimiento del negocio.

Una correcta gestión de pruebas implica una correcta gestión de la calidad.

En el caso de aplicaciones web, es a veces habitual sacar al mercado el software lo antes posible (para que no nos “pisen” nuestros competidores). Los errores pasan a un segundo plano en la primera versión y será en versiones posteriores cuando se ofrezca la aplicación mejorada. Este modelo tiene, lógicamente, el defecto de perder calidad de la marca de tu empresa y los clientes pueden desconfiar y recordarte por lo mala que fue tu primera versión.

Los defectos se introducen, en su mayoría, durante la fase de análisis de requisitos y diseño, pero suelen ser detectadas durante las pruebas de aceptación y en la explotación (que son las fases donde resulta más costosa económicamente su reparación).

Para saber más

¿Qué supone para tu negocio un error en el software? Relación entre pruebas y calidad del software.

1.3.- Estrategias de pruebas.

Caso práctico

Ada tiene definida en BK una estrategia de pruebas por la cual todo proyecto debe pasar antes de llegar a manos del cliente. Hoy, le toca a **María** explicarles a sus compañeros cuál será la estrategia a seguir para el nuevo proyecto que BK tiene asignado.



pexels.com. *Reunión de proyecto* (Licencia Pexels)

Normalmente, un proyecto software será sometido a una estrategia de pruebas. La estrategia utilizada será la que mejor se adecúe a nuestro proyecto.

Todos los desarrollos de software tienen una característica en común: lo desarrollan personas. Las personas cometemos errores y cambiamos de ideas, por eso hay que probar y volver a probar.

El coste económico relativo a un error se multiplica por un factor de 1 si se comete en la fase de análisis de requisitos, por 3-6 si es en la fase de diseño, por 10 si es en la codificación y por 40-50 si es en la fase de pruebas.

Es muy importante contar con el apoyo de la dirección del proyecto y, en su caso, por la dirección de la empresa. Deben formar parte del presupuesto, la planificación y la asignación de recursos del proyecto.

Preguntas para determinar la estrategia de pruebas a seguir: ¿Cuántos niveles de pruebas serán necesarios planificar? ¿Cuándo será conveniente parar? ¿Será mejor automatizar las tareas? Etc. Por ello, **el objetivo de la estrategia de pruebas es determinar los tipos de pruebas a realizar, el calendario de las mismas, los responsables, los recursos asignados y el alcance de las pruebas**.

Además, será necesario conocer:

- La complejidad de la aplicación y de los módulos que la forman.
- Plataforma en la que va a funcionar la aplicación.
- Normativa legal pertinente.
- Conocimientos y experiencia de los responsables de realización de las pruebas.

La estrategia de pruebas es un proceso complejo que consta de tres partes:

1-Visión general de la estrategia de pruebas

Según el tamaño o complejidad de la aplicación, se definen cómo:

- Las pruebas que se van a realizar.
- Las diferentes versiones de las pruebas.
- La descripción del objetivo que se pretende alcanzar en cada una de ellas

2-Tareas en la realización de la prueba

Consiste en describir:

- Las tareas propias a realizar.
- El ambiente en el que serán realizadas.
- Los datos de entrada de las pruebas y la finalización de las mismas, describiendo las acciones que deben ser ejecutadas cuando las pruebas finalicen.

3-Resultados

Consiste en documentar y revisar los resultados esperados (los que deberían de haber pasado) y los documentos reales (lo que ha pasado realmente). Si un caso de prueba falla, debe de quedar reflejado el defecto detectado.

Reflexiona

El coste medio de la fase de pruebas está en torno al 40-50 % del coste total del proyecto.

1.4.- Limitaciones del proceso.

Caso práctico

Juan no puede evitar pensar que el proceso de pruebas va a llevarles un tiempo precioso, tiempo que podría emplear para avanzar en otras fases del proyecto… ¿Por lo menos nos servirá para encontrar todos los errores?

Resulta que, por su propia naturaleza, no es posible probar completamente los productos software, tanto por su complejidad como por su coste.

Por tanto, el propósito más importante es el de reducir, mediante la detección temprana de problemas, los riesgos derivados de la aparición de los posibles defectos en los procesos de implantación y explotación.

En la práctica se considera que, en software, el conjunto completo de casos de prueba es teóricamente infinito. El número de pruebas a realizar implicará un equilibrio entre recursos y tiempo.

Según los datos y tal como demuestra la experiencia, resulta que por muy cuidadosos que hayamos sido en la estrategia de pruebas siempre habrá una serie de errores que sólo aparecen cuando el cliente comienza a usarlo. Debemos tener claro que "el cliente siempre lleva la razón" y, por esta razón, después de un cierto tiempo en el que el cliente haya estado usando nuestra aplicación y descubra defectos, o que el programa no realice lo que se esperaba (o en la forma en que se esperaba), debemos revisar la aplicación y corregir esas taras.

Es por ello que las pruebas de aceptación, como veremos más adelante, tienen gran importancia y pueden llevarse a cabo durante incluso semanas después de haber entregado el proyecto al cliente.

Las empresas se sorprenden de que no se pueda estimar de forma realista el coste de sus proyectos. El coste asociado al proceso de pruebas no es fácilmente estimable, puesto que no hay dos proyectos iguales.



Pixabay.com. *Coste de proyecto* (Licencia Pixabay)

El software no puede ser 100 % libre de errores. La cuestión más difícil en la realización de pruebas es estimar el momento en que debemos parar.

Por propia definición, un proyecto de software siempre tendrá defectos y nuestra principal finalidad es minimizarlos. Para conseguirlo, hay que tener en cuenta una serie de principios:

Falta de conocimiento

Siempre hay falta de conocimiento del estado real de las aplicaciones durante el desarrollo de los proyectos. (Este es el motivo principal por el que nunca sabremos el coste asociado a la estrategia de pruebas).

Sin garantía

Tener buenos conocimientos técnicos no son una garantía de éxito de un proyecto.

Estándares

Hay que seguir procedimientos y actividades estandarizadas que nos guíen durante el desarrollo del proyecto y nos permitan realizar los mismos de forma repetitiva.

2.- Tipos de pruebas.

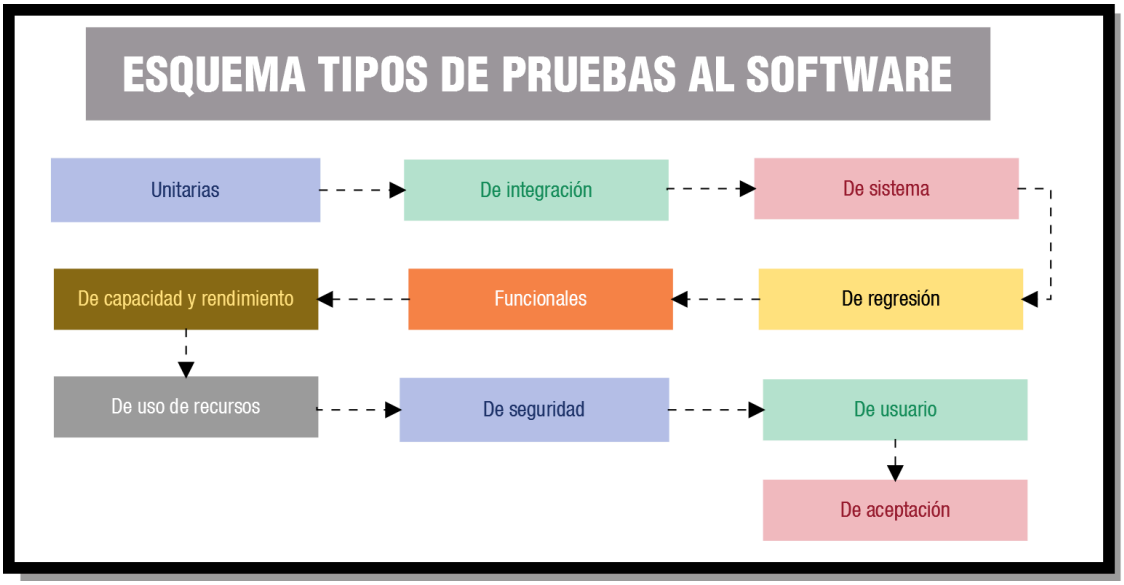
Caso práctico

Aún con sus limitaciones, las pruebas tienen ventajas incuestionables. Pero, ¿qué tipos de pruebas hay que hacerle a la aplicación?, pregunta **Ana**.

"Hay muchos tipos de pruebas y todas son muy importantes." contesta **María**. Vamos a ver cuántos tipos hay y qué función principal tiene cada uno

Normalmente un sistema es sometido a una serie de pruebas para determinar que el software cumple la función para la cual ha sido construido y desarrollado. Según la bibliografía que consultes, encontrarás diferentes criterios a la hora de clasificar los distintos tipos de pruebas al software. La clasificación más habitual es la que comprende las siguientes de la siguiente figura.

A lo largo de la presente unidad, vas a ir viendo qué objetivos se persiguen en cada tipo de pruebas y cómo se debe planificar para que los resultados sean lo más eficientes posibles.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Los tipos de pruebas al software se pueden clasificar en:

Unitarias

Se realizan a módulos o clases del programa, por separado. Es decir, a cada unidad que compone la aplicación. Para hacerlas, debemos de tener en cuenta:

- La interfaz del módulo o componente.
- El impacto de datos globales sobre el módulo.
- Las estructuras de datos en el módulo.
- Las condiciones límite.
- Los distintos caminos de ejecución de las estructuras de control.

De integración

Integran componentes y módulos del programa según un orden preestablecido. Se prueba el funcionamiento de la interrelación de todos los módulos.

De sistema

Validan que la aplicación tenga la funcionalidad que el usuario final espera de ella. Las pruebas de sistema se ven como una "caja negra".

De regresión

Después de realizar algún tipo de modificación en el código del programa, consisten en volver a ejecutar un conjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

Funcionales

Su objetivo es detectar errores en la implementación de los requerimientos.

De capacidad/rendimiento

La prueba de capacidad (también conocida como prueba de resistencia) ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales. La de rendimiento determina los tiempos de respuesta (lo rápido que realiza una tarea un sistema en condiciones de trabajo), el espacio que ocupan los módulos en disco y en memoria, el flujo de datos que se genera, etc.

De uso de recursos

También conocidas como pruebas de eficiencia, se encargan de optimizar el uso de los recursos software y hardware de una aplicación.

De seguridad

La prueba de seguridad intenta verificar que los mecanismos de protección incorporados en nuestro sistema lo protegen de accesos no autorizados.

De usuario

Este tipo de prueba (también conocida como de usabilidad) se refiere a asegurar que la interfaz de usuario (GUI) sea amigable, intuitiva y que funcione correctamente.

De aceptación

Estas pruebas son realizadas por el cliente final. Consisten en pruebas funcionales sobre el sistema completo. Se realizan sobre el producto terminado y nunca durante el desarrollo del sistema o de la aplicación.

Autoevaluación

Relaciona los tipos de pruebas al software con su característica más relevante, escribiendo el número asociado a la característica en el hueco correspondiente.

Ejercicio de relacionar

Tipo de prueba	Relación	Características
Prueba de Regresión.	0	1. Se prueba la interrelación de todos los módulos.
Prueba de Integración.	0	2. Las realiza el cliente sobre el producto terminado.
Prueba de Aceptación.	0	3. Se prueba la no existencia de errores tras una modificación.

Enviar Habilitar JavaScript

La prueba de integración asegura que funciona la conexión entre todos los módulos que componen la aplicación. La de regresión sirven para comprobar que un cambio no introduce errores colaterales en el sistema y la de aceptación es la prueba final, donde el cliente comprueba que la aplicación funciona y, además, de la manera en que éste había requerido desde un principio.

2.1- Integración.

Caso práctico

Antonio, que está estudiando el ciclo a distancia, sabe que hay que probar cada módulo del programa, de forma independiente, y comprobar que no tiene errores.

"¿Eso es todo?" Le pregunta **María**. ¿La relación entre varios módulos puede dar errores, aunque los módulos, probados de uno en uno, no los hayan dado?

Las pruebas de integración son aquellas que se realizan una vez probado el funcionamiento de las partes o módulos que componen la aplicación por separado, es decir, una vez concluidas las pruebas unitarias.



Pixabay.com. *Pruebas de integración* (Licencia Pixabay)

Consiste en verificar que el software, en conjunto, cumple su misión y se realiza sobre la unión de todos los módulos a la vez, para probar que la interrelación entre ellos no da lugar a ningún error o defecto. Son muy importantes por el hecho de que los módulos que componen una aplicación suelen fallar cuando trabajan de forma conjunta, aunque previamente se haya demostrado que trabajan bien de manera individual.

La integración recomendable es la llamada **incremental**, en la cual se van añadiendo nuevos módulos cada vez en la realización de las pruebas (y no todos de golpe). El nuevo módulo

incorporado se debe probar con el conjunto de módulos que ya han sido probados.

La prueba de integración verifica la interacción entre componentes del sistema

Dependiendo de la filosofía de integración, se definen dos técnicas: **ascendentes y descendentes**.

Para saber más

Diferencias entre pruebas unitarias y pruebas de integración: https://ricardogeek.com/la-diferencia-entre-pruebas-unitarias-y-de-integracion/

2.1.1.- Ascendentes.

Es un tipo de estrategia de pruebas de integración en el cual se comienza por los módulos de más bajo nivel hasta llegar al programa principal. Es decir, se prueba la relación entre pocos módulos y, si es correcta, se va ascendiendo de manera que cada vez habrá más módulos involucrados, llegando finalmente a probar la aplicación en su totalidad. La integración es de abajo a arriba.



Pixabay.com. *Resolución de puzle* (Licencia Pixabay)

El principal inconveniente es que tenemos una gran incertidumbre hasta el final de la prueba, ya que el programa en sí no existirá hasta que no se añada el último módulo. Como ventaja, no existe la necesidad de operar con resguardos.

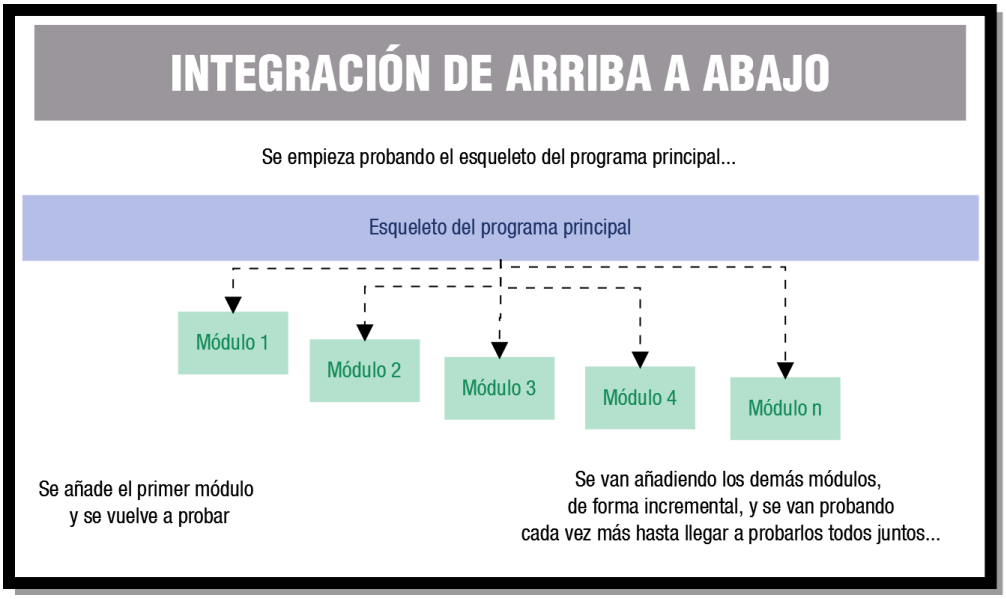
A modo de resumen, las características más importantes de la estrategia incremental ascendente son:

- Se combinan los módulos de bajo nivel en grupos con una función similar.
- Se controlan las entradas en esos módulos de bajo nivel.
- Se prueba todo el grupo.
- Se reemplazan los controladores y se combinan los grupos moviéndose hacia arriba por la estructura de módulos hasta llegar al programa principal.

Resguardo es una estructura general prevista para un módulo en particular, utilizado cuando no se conoce de antemano el código del módulo real.

2.1.2.- Descendentes.

Es la otra estrategia de las pruebas de integración.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Consiste en integrar los módulos moviéndose de arriba a abajo por la jerarquía, comenzando desde el esqueleto del programa principal, hasta ir incorporando los módulos subordinados de forma incremental. Tras la incorporación de cada módulo, habrá que realizar la prueba de integración. El programa principal actuará como coordinador de la prueba.

La integración es, por tanto, de arriba a abajo.

Ventaja

Se prueban antes los módulos más importantes.

Inconveniente

Como desventaja, es necesario trabajar con resguardos (con las complicaciones que eso conlleva).

A modo de resumen, la estrategia incremental descendente comprende los siguientes pasos:

- Se usa el módulo principal como controlador de la prueba, disponiendo de resguardos para todos los módulos subordinados.
- Se van sustituyendo los resguardos subordinados, uno a uno, por los módulos reales.
- Cada vez que se integra un módulo se realizan las pruebas.

- Tras terminar con éxito las pruebas, se reemplaza otro resguardo por otro módulo real.

Debes conocer

El orden de integración elegido (ascendente o descendente) afecta a diversos factores, como son:

- Las herramientas necesarias.
- El coste de preparación de los casos de prueba.
- El orden de codificar y probar los módulos.
- El coste de la depuración.
- La forma de preparar los casos de prueba.

La selección de una estrategia de integración depende de las características del software y del plan de proyecto Es bastante habitual combinar las dos técnicas: usar la integración descendente para los módulos superiores de la estructura del programa y la ascendente para los niveles subordinados de bajo nivel.

Autoevaluación

Pregunta

Tras probar varios módulos, de forma independiente, terminamos por librarlos de errores. ¿Es necesario realizar la prueba de integración?

Respuestas

No.

Sí.

Retroalimentación

Incorrecta. El funcionamiento de cada módulo por separado no garantiza la ausencia de errores en su interrelación.

Efectivamente, hay que realizarla.

Solución

1. Incorrecto
2. Opción correcta

2.2.- Sistema.

Caso práctico

Llegados a este punto. ¿Podemos asegurar que una aplicación probada para un equipo y software determinado va a funcionar también en otro diferente? Es decir, ¿será la aplicación totalmente portable entre distintas plataformas hardware y software?

Las pruebas de sistema verifican el comportamiento del sistema en su conjunto. En concreto, se comprueban los requisitos no funcionales de la aplicación:

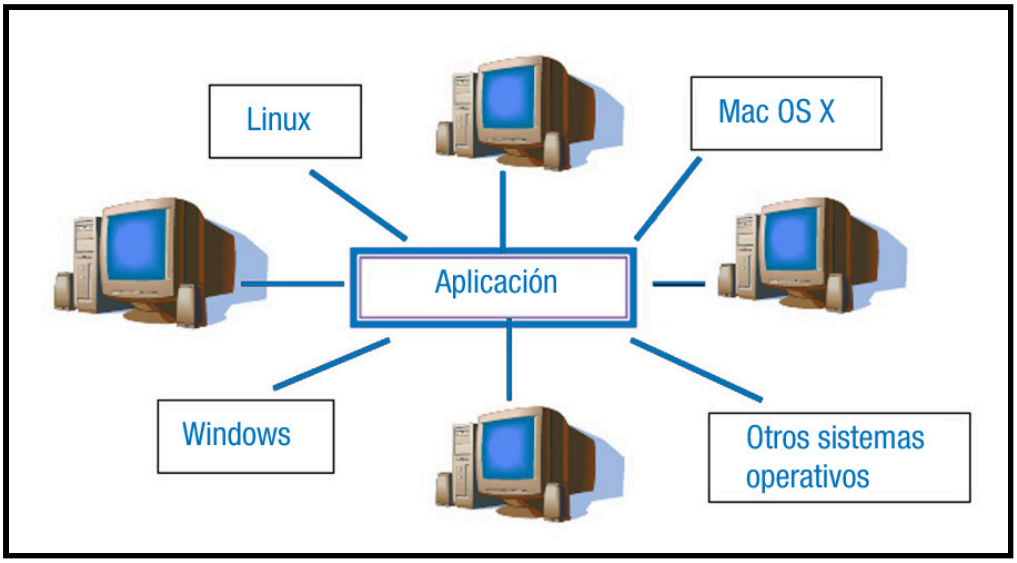
- Seguridad.
- Velocidad.
- Exactitud.
- Fiabilidad.

También se prueban los interfaces externos con otros sistemas, utilidades, unidades físicas y el entorno operativo.

Entre las pruebas de sistema más relevantes, encontramos las que vemos a continuación.

2.2.1.- Configuración.

El objetivo es verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas y determinar la configuración óptima del sistema.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Estas pruebas verifican la instalación del software en el entorno de destino y comprueban el comportamiento del sistema frente a los requisitos de configuración. Además, analizan el software bajo configuraciones diferentes para diferentes usuarios.

La meta de la prueba es hacer que la aplicación falle en el cumplimiento de los requerimientos de configuración, de manera que los defectos escondidos sean identificados, analizados, arreglados y prevenidos en el futuro. Durante estas pruebas, la persona encargada de la realización de pruebas funcionales al proyecto de software, valida que nuestro proyecto actual es capaz de soportar diferentes tipos de tecnologías de hardware, como diferentes tipos de impresoras, interfaces, topología, etc.

Estas pruebas también son llamadas pruebas de hardware o pruebas de portabilidad.

Actualmente se simulan las pruebas de configuración en equipos con máquinas virtuales, siendo su realización totalmente equivalente a la que se haría en varios equipos físicos.

En general, lo que se suele hacer en esta prueba es configurar una sala o laboratorio con varios equipos físicos que ejecuten diferentes combinaciones de sistemas operativos, exploradores web y otro software, para probar la aplicación con distintas combinaciones de hardware y software. Esta realización (manual) puede resultar bastante costosa, tanto en tiempo como en recursos, por lo que en la actualidad está automatizada con herramientas software que simulan todas estas situaciones sin tener que llevarlas a cabo físicamente.

Autoevaluación

Pregunta

El objetivo fundamental de la prueba de configuración es determinar el funcionamiento óptimo del sistema en base al uso eficiente de sus recursos.

Respuestas

Sí, ese es el objetivo fundamental.

No, el objetivo de la prueba no es ese.

Retroalimentación

No es correcto, lo que se mide es la capacidad de funcionamiento óptimo en base a distintos entornos de configuración de hardware y software.

Efectivamente, eso no es lo que se mide en esta prueba.

Solución

1. Incorrecto
2. Opción correcta

2.2.2.- Recuperación.

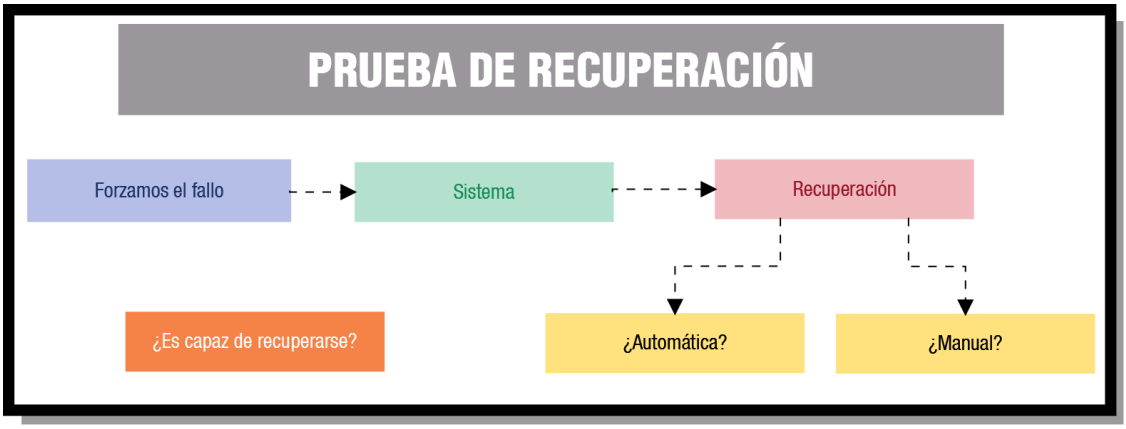
La prueba de recuperación es una prueba de sistema que fuerza el fallo del software de muchas formas y verifica que la recuperación se lleva a cabo de forma satisfactoria.

Es un parámetro muy importante de la aplicación, porque si un sistema no es capaz de recuperarse ante una entrada no válida o de un fallo súbito, la calidad de nuestro software quedará en entredicho.

La recuperación de nuestra aplicación puede llevarse a cabo de forma automática o manual. Si la recuperación es automática hay que evaluar la corrección de:

- La inicialización.
- Los mecanismos de recuperación del estado del sistema.
- Los datos.
- El proceso de arranque.

Si la recuperación requiere la intervención humana, hay que evaluar los tiempos medios de reparación (TMR) para determinar si están dentro de unos límites aceptables.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Un sistema tolerante a fallos evita que cese el funcionamiento de todo el sistema cuando se produce un fallo del proceso.

Autoevaluación

Pregunta

Las pruebas de sistema se centran en verificar los requisitos no funcionales de la aplicación. ¿Cuáles son?

Respuestas

Velocidad, seguridad, exactitud y funcionalidad.

Velocidad, seguridad, exactitud y fiabilidad.

Validación, velocidad, seguridad y fiabilidad.

Retroalimentación

Incorrecto. La funcionalidad no es un requisito no funcional.

Así es. Vas captando la idea.

No es del todo correcta. La validación no es un requisito no funcional.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

2.3.- Regresión.

Caso práctico

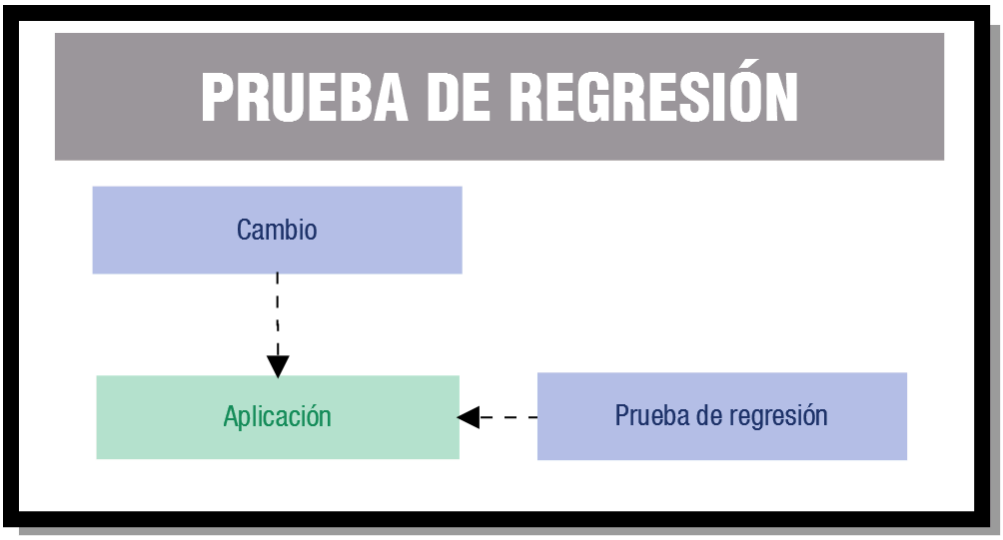
Después del éxito de la prueba de integración, **Juan** se percata de que falta un módulo en la aplicación. Lo diseña y lo añade.

"¿Habrá que hacer otra vez todas las pruebas?" **Juan** espera que no. Sin embargo…

Resulta que cada vez que se añade un nuevo módulo a la aplicación, o cuando se modifica, el software cambia.

Como resultado de esa modificación (o adición) de módulos, podemos introducir errores en el programa que antes no teníamos. Por ello, se hace necesario volver a probar la aplicación. La prueba de regresión es volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

Así, el objetivo de las pruebas de regresión es comprobar que los cambios sobre un componente de una aplicación no introducen errores adicionales en otros componentes no modificados.



Ministerio de Educación y Formación Profesional (Elaboración propia)

La prueba de regresión se puede hacer manualmente, volviendo a realizar un subconjunto de todos los casos de prueba o utilizando herramientas automáticas (es lo más frecuente).

El conjunto de pruebas de regresión contiene, a su vez, tres clases de prueba diferentes:

- Planificar una muestra de pruebas sobre todas las funciones del software.
- Plantear pruebas adicionales que se centren en las funciones del software que se van a ver afectadas por el cambio.
- Planificar pruebas que se centren únicamente en los componentes que han cambiado.

Es totalmente desaconsejable, por razones obvias de pérdida de tiempo, volver a realizar todas y cada una de las pruebas sobre cada uno de los componentes.

Actualmente, las pruebas de regresión son una parte integral del método de desarrollo de software conocido como Programación Extrema (Extreme Programming). Se utilizan herramientas que permiten detectar este tipo de errores de manera parcial o totalmente automatizada en cada una de las fases del desarrollo del software.

Debes conocer

Entre las herramientas libres para automatizar pruebas de regresión para Java encontramos JUnit, JWebUnit, NUnit, etc. En los siguientes apartados de la presente unidad profundizaremos más sobre la automatización de las pruebas, tanto de regresión como el resto.

2.4.- Funcionales.

Caso práctico

"He oído hablar de las pruebas de caja negra", le comenta **Ana** a **María**. "¿A qué se refieren?"

Pruebas de caja negra son las que determinan las salidas que se esperan frente a unas entradas concretas. Son la base de las pruebas funcionales, donde sólo nos interesa que se cumplan los requerimientos, y no la forma en que se han implementado.

Las pruebas funcionales (o de conformidad) validan si el comportamiento observado del software es conforme con sus especificaciones (requisitos funcionales).

También son conocidas como Functional Testing y su objetivo fundamental es probar que las aplicaciones desarrolladas cumplen la función específica para la cual ha sido creada.

Normalmente, los responsables de realizar estas pruebas son los analistas de la aplicación (las personas responsables de la especificación de los requerimientos) con apoyo de los usuarios finales. Su aprobación dará paso a la fase de producción propiamente dicha.

Son **pruebas de caja negra** y las personas encargadas de realizar las pruebas, no les importa cómo se generan las respuestas, sólo analizan las salidas de la aplicación frente a sus entradas.



freepik.es. *Pruebas de software* (Licencia Freepik)

Las pruebas funcionales intentan responder a las preguntas: "¿funciona esta utilidad de la aplicación?", "¿el usuario podrá hacer esto?".

Los sistemas que ya han sido sometidos a pruebas unitarias requieren menor tiempo para las pruebas funcionales. Esto es así porque se considera que el sistema debe ser ya bastante estable, con pocos errores críticos. Si esto no se cumpliese, deberíamos volver hacia atrás, a la etapa de pruebas unitarias (un sistema inestable sometido a pruebas funcionales requiere altos tiempos asociados con un avance lento, además al ser pruebas de caja negra, a la persona encargada de realizar las pruebas, no le interesa el funcionamiento intermedio del sistema.

En este tipo de pruebas no interesa el código, ni el algoritmo, ni la eficiencia ni la construcción de elementos innecesarios. Sólo interesa verificar que las salidas que devuelve la aplicación son las esperadas, en función de las entradas que tenga.

Dentro de las pruebas funcionales, encontramos tres tipos:

Análisis de valores límite

Como entradas, seleccionamos aquellos valores que están en el límite donde la aplicación es válida. (Si estos valores no dan problemas, el resto tampoco los dará).

Particiones equivalentes

Como entradas, se seleccionará una muestra representativa de todas las posibles. El resultado será extrapolable al resto.

Pruebas aleatorias

Como entradas, se selecciona una muestra de casos de prueba al azar. Se utiliza sólo en aplicaciones no interactivas.

Autoevaluación

Pregunta

Una de las estrategias en el diseño de pruebas funcionales, de caja negra, es la de particiones variables. ¿Cuál es su principal característica?

Respuestas

Selección de los valores límite de validez de las entradas.

Escoger al azar una muestra de entradas.

Escoger una muestra representativa de entradas.

Retroalimentación

Incorrecto. Esa estrategia es la de valores límite.

No es del todo correcto. Ese sistema es el de pruebas aleatorias.

Muy bien, así es.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

2.5.- De capacidad y rendimiento

Caso práctico

¿Qué pasará en los momentos en que las solicitudes a la aplicación se multipliquen? ¿Será capaz de aguantar estas cargas anormales? ¿Cuál será el tiempo de respuesta de la aplicación?

"Vamos a probarlo", asevera **María**.

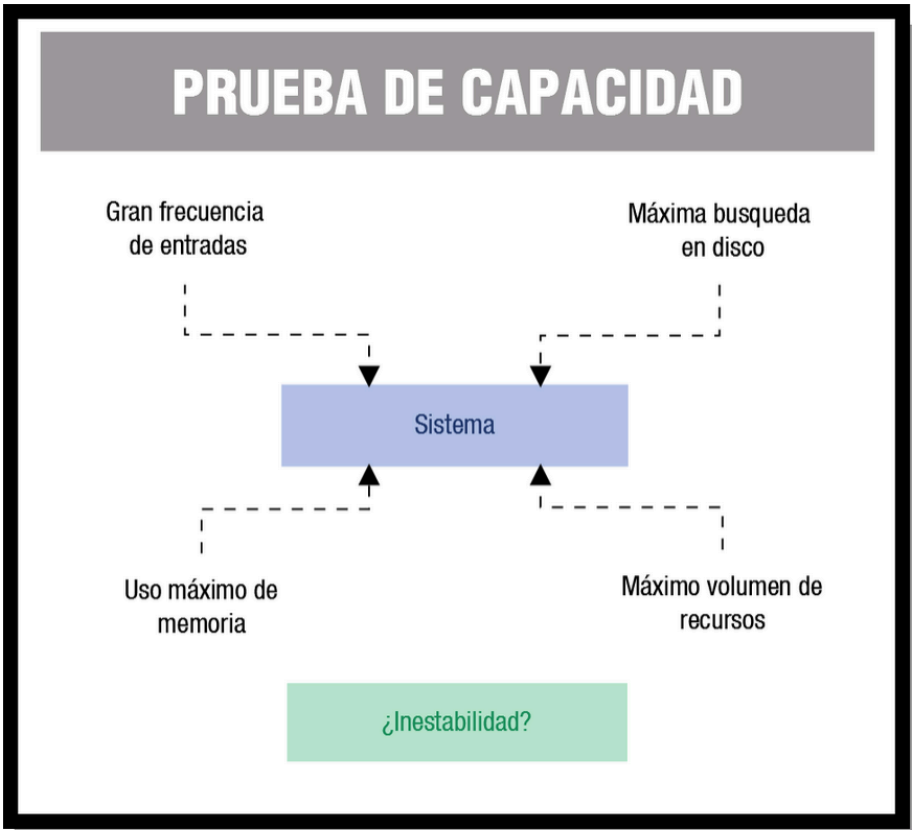
A la hora de medir los límites del sistema, es importante diferenciar entre las pruebas de capacidad y las pruebas de rendimiento:

Pruebas de capacidad

La prueba de capacidad (también conocida como prueba de resistencia o stress) ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales. Es decir, determina hasta dónde el sistema es capaz de soportar determinadas condiciones extremas.

También tiene como finalidad determinar la capacidad del programa para soportar entradas incorrectas.

Para realizar la prueba de capacidad se recurre a los siguientes métodos: Aumentar la frecuencia de entradas para ver cómo responde la aplicación. Se ejecutan casos que requieran el máximo de memoria. Se buscan una cantidad máxima de datos que residan en disco. Se realizan pruebas de sensibilidad, intentando descubrir combinaciones de datos que puedan producir inestabilidad en el sistema.



Ministerio de Educación y Formación Profesional (Elaboración propia)

Pruebas de rendimiento

Determinan los tiempos de respuesta (lo rápido que realiza una tarea un sistema en condiciones de trabajo), el espacio que ocupan los módulos en disco y en memoria, el flujo de datos que genera a través de un canal de comunicaciones, etc. La prueba de rendimiento está diseñada para probar el rendimiento del software en tiempo de ejecución dentro del contexto de un sistema integrado. Se realiza durante todos los pasos del proceso de la prueba. Incluso a nivel de unidad, se debe asegurar el rendimiento de los módulos individuales a medida que se llevan a cabo las *pruebas de caja blanca* (en donde se evalúan las salidas en función de las entradas. No se evalúa la estructura interna de la aplicación). Sin embargo, hasta que no están completamente integrados todos los elementos del sistema no se puede asegurar realmente el rendimiento del sistema.

Prueban el rendimiento del software en tiempo de ejecución dentro del contexto de un sistema integrado.

- Requiere de instrumentación tanto software como hardware para los procesos de monitorización (es el proceso que asegura que nuestro proyecto va encaminado eficazmente hacia el resultado esperado) y medición.
- Se lleva a cabo durante todos los pasos de prueba.

Cuanto más se tarde en detectar un defecto de rendimiento, mayor será el coste de la solución.

Los **sistemas de tiempo real** (son aquellos al que le exigen respuestas bajo restricciones de tiempo) y los **sistemas empotrados** (diseñados para realizar determinadas funciones dedicadas, frecuentemente en un sistema de tiempo real) se tienen que ajustar a los requisitos de rendimiento.

2.6.- De uso de recursos.

Caso práctico

¿Será suficiente con cierta capacidad de memoria RAM para que un equipo soporte la aplicación? ¿Cuáles serán los demás requisitos del sistema?

"Creo que es importante, en toda aplicación, determinar los requisitos mínimos del sistema para que la aplicación funcione. Vamos a probar la eficiencia de la aplicación.



pixabay.com. *Interconexión* (Licencia Pixabay)

La evolución de los sistemas informáticos lleva incorporada una mayor complejidad en su diseño y en los recursos hardware y software que requiere. Las aplicaciones requieren cada vez mayor memoria RAM y espacio en disco para funcionar correctamente y nuestros equipos habitualmente tienen limitaciones en este sentido. Es por ello importante el uso eficaz que hace una aplicación de los recursos de que dispone.

La prueba de uso de recursos es también conocida como prueba de eficiencia. Se desarrolla un conjunto de técnicas que garantice un uso eficiente de los recursos.

Este software estará diseñado, implementado y optimizado sobre sistemas concretos. Igualmente, existe la necesidad de disponer de técnicas que aseguren el uso en sistemas que no son los originales en los que la aplicación fue diseñada. Es decir, asegurar la adaptación a otros sistemas.

La optimización es necesaria para lograr un uso eficiente de recursos. La optimización se debe hacer tanto en el código como en el uso del código. Todo ello para lograr la adaptación

del software a la arquitectura de destino y así reducir los tiempos de ejecución de forma automática.

Autoevaluación

Pregunta

¿Es totalmente imprescindible tener conocimiento de antemano de la arquitectura del computador destino de nuestra aplicación para hacer un uso eficiente de recursos hardware?

Respuestas

Sí.

No.

Retroalimentación

Incorrecta. El sistema debe ser eficiente en distintas configuraciones, adaptándose a las mismas.

Efectivamente, aunque es aconsejable no es imprescindible.

Solución

1. Incorrecto
2. Opción correcta

Reflexiona

En la actualidad, la preocupación por la eficiencia en cuanto a recursos de la máquina (tiempo de procesador, consumo de memoria…) ha dejado paso a la preocupación por la usabilidad de la aplicación (forma en la que la aplicación es presentada al cliente).

Quizás la eficiencia en el uso de recursos sea algo más importante en el caso de aplicaciones web o en sectores críticos que despliegan sistemas de control en tiempo real, permitiendo a los desarrolladores concentrarse en aspectos de usabilidad y accesibilidad del software.

2.7.- De seguridad.

Caso práctico

Ana, que está "empapándose" de todo lo que acontece, cae en la cuenta de que la aplicación puede ser vulnerable frente a accesos no autorizados. Recibe la felicitación de **María**.

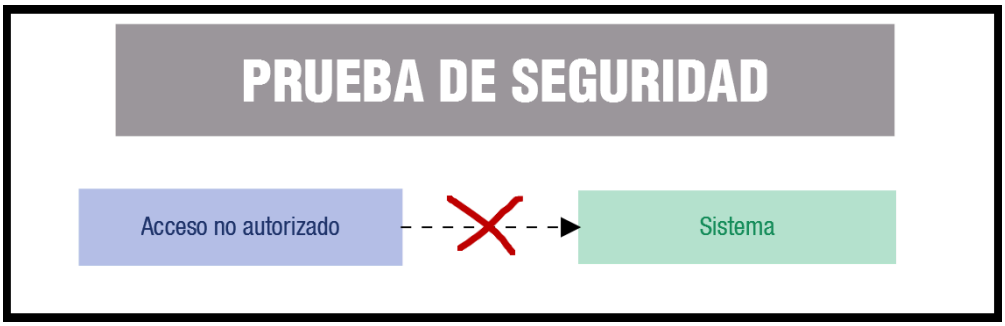
"Muy bien. Ha llegado el momento de probar si nuestra aplicación protegerá de aquellos que intenten acceder a ella sin las claves necesarias".



pixabay.com. *Ciberseguridad* (Licencia Pixabay)

La prueba de seguridad (también conocida como prueba de Integridad) intenta verificar que los mecanismos de protección incorporados en el sistema lo protegerán de accesos no autorizados. Mide la habilidad de un sistema para soportar ataques (tanto accidentales como intencionados) contra su seguridad. El ataque se puede ejecutar mediante los programas, los datos o los documentos de la aplicación.

Es decir, determinan los niveles de permiso de usuarios, las operaciones de acceso al sistema y acceso a los datos.



Ministerio de Educación y Formación Profesional (Elaboración propia)

En concreto, se intenta verificar que el sistema es robusto frente a problemas de seguridad, tales como:

- Intentar conseguir las claves de acceso de cualquier forma.
- Atacar con software a medida.
- Bloquear el sistema.
- Provocar errores del sistema, entrando durante su recuperación.

Hay que tener especial cuidado con las entradas no autorizadas, especialmente cuando en el sistema se maneja información de carácter privado y personal que puede atentar contra la intimidad de las personas.

Durante la prueba, lo usual es que el encargado desempeñe el papel de intruso y trate de violar los mecanismos de seguridad de acceso al sistema, intentando obtener la clave de acceso de cualquier forma. Además, también hará lo posible para bloquear el sistema, curiosear los datos públicos en busca de claves e introducir errores de forma consciente al sistema para ver cómo responde éste.

Autoevaluación

Pregunta

En la prueba de seguridad, no se considera como potencial problema:

Respuestas

Bloquear el sistema.

La tardanza en la respuesta de la aplicación.

Intentar conseguir las claves de acceso.

Acceder a los datos.

Retroalimentación

Incorrecta, bloquear el sistema es un aspecto que hay que revisar en esta prueba.

Efectivamente, este aspecto carece de interés en la prueba de seguridad.

No es la opción correcta. Es uno de los principales riesgos que hay que prevenir.

No es correcta, no puede darse un acceso no autorizado a los datos de la aplicación.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

Para saber más

Pentesting - Análisis integral del estado de seguridad

2.8.- Manuales y automáticas.

Caso práctico

"Por favor, esto es una locura", dice Antonio. "¿Hay herramientas que automaticen todo esto? No habrá que ir haciendo prueba a prueba manualmente… ¿no?" "Claro que sí", responde Juan. "Ahora verás unas cuantas."

Ya sabes que la prueba consiste en la ejecución de un programa con el único objetivo de encontrar defectos. Las condiciones bajo las cuales se realizará la prueba tienen que estar prefijadas para poder evaluar los resultados que obtengamos.

Es decir, las pruebas no se hacen al azar, sino que seguirán una estrategia. En ella, será necesario definir una serie de casos de prueba, que son un conjunto de entradas, de condiciones de ejecución y resultados esperados, desarrollados para un objetivo particular.



pixabay.com. *Éxito* (Licencia Pixabay)

Todos los tipos de pruebas que estamos viendo se ejecutan de manera regular con el objetivo de probar que la evolución del software desarrollado funciona perfectamente, tanto para las funcionalidades nuevas como para aquellas que ya existían. Una de las claves es que las pruebas sean automatizadas, porque si se ejecutaran manualmente, el coste de hacerlo "continuamente" sería inasumible.

¿Por qué automatizar?

La razón obvia es ganar tiempo y facilitarnos la vida.

Aparte de esta, que es evidente, los otros motivos son:

- Mejorar la calidad del producto.
- Detectar errores en etapas tempranas del desarrollo del proyecto.
- Disminuir el tiempo de pruebas y, por tanto, el tiempo de salida al mercado.
- Reducir costes.
- Ejecutar pruebas de manera continua.

Herramientas

Es importante, en caso de utilizar una herramienta que automatice el proceso de pruebas, realizar una correcta selección de la misma. En el mercado disponemos de una amplia gama de herramientas en este sentido. Existen infinidad de herramientas gratuitas que, según las características de nuestro proyecto (número de requisitos, grado de estabilidad, organización del equipo, etc.), podemos valorar muy positivamente.

Según Meudec, las herramientas de pruebas de software persiguen automatizar tres tareas fundamentales:

1. Tareas administrativas y generación de documentación.
2. Tareas mecánicas (ejecución).
3. Tareas de generación de casos de prueba.

En los últimos años se han desarrollado una serie de *frameworks* (plataforma, entorno, marco de trabajo rápido de desarrollo de aplicaciones) (como el conocido entorno JUnit) para automatizar pruebas unitarias y hay una multitud de entornos de desarrollo (como NetBeans) que incorporan *plugins* (son complementos de aplicaciones que añaden una funcionalidad nueva y, generalmente, muy específica) y componentes que permiten su integración.

La elección de la herramienta de automatización es muy importante y depende del tipo de pruebas a automatizar, del entorno de desarrollo utilizado y del lenguaje de programación con el que hayamos codificado la aplicación.

2.8.1.- Herramientas software para la realización de pruebas.

A continuación, se presenta una tabla con varias herramientas que permiten la automatización de ciertos tipos de pruebas de software.

Herramientas de automatización de pruebas software

Tipo de prueba	Herramienta	Licencia	Lenguaje
Unitarias	**JUnit**	CPL	Java
	Simple Test	LGPL	PHP
	PHP Unit	PHP	PHP
	Jtiger	GPL	Java
Funcional y de Regresión	Selenium	Apache	Java, PHP, Python, Ruby, entre otros.
	Watir	BSD	Ruby
	Watij	GPL v2	Java
	JWebUnit	GPL	Java
	NUnit	GPL	C#, J#, VB y C++
	Http Unit	GPL	Java
Integración	TestNG	GPL	Java
	Hudson	Subversion	Java
	Continuum	Subversion	Java
Carga y Rendimiento	JMeter	Apache	Java
	Jcrawler	CPL	Java
Aceptación	FitNesse	GPL	Java, PHP, Ruby, .NET
	Concordion	GPL	Java, Python, Ruby, .NET
	Nessus	GPL	NASL,XML,HTML

En la automatización de las pruebas al software, debes tener en cuenta:

- **Definir los objetivos de la automatización en base a los objetivos de calidad.**
- **Elegir las pruebas que se van a automatizar.**
- **Seleccionar correctamente las herramientas para la automatización.**
- **Se trata de una inversión cuyos beneficios se observan a medio plazo.**
- **Debe contar con personal especializado.**

2.8.2.- JUnit

JUnit es un paquete Java utilizado para automatizar los procesos de prueba. Mediante la creación de Tests, JUnit realizará una prueba en el código que indique el usuario.

Principios F.I.R.S.T. para el desarrollo de pruebas

FIRST es el acrónimo de las cinco características que deben tener nuestros tests unitarios para ser considerados tests con calidad:

- F: Fast, los test se han de ejecutar rápidamente.
- I: Isolated, los test se realizan sobre una clase sin interacción con otras.
- R: Repeatable, el orden de ejecución de los test no debe influir en el resultado final.
- S: Self-validating, los test se han de ejecutar de modo automático.
- T: Timely, se han de crear al mismo tiempo que el software que se está creando.

¿Por qué utilizar JUnit?

JUnit es un paquete Java utilizado para automatizar los procesos de prueba. Mediante la creación de Tests, JUnit realizará una prueba en el código que indique el usuario.

Para poder aplicar JUnit en un proyecto, es necesario hacer un repaso por sus principales características, teniendo en cuenta que cumple los principios F.I.R.S.T. del apartado anterior.

¿Por qué utilizar JUnit?

- Los test JUnit permiten asegurar la calidad del software programado.
- Los test JUnit chequean los resultados esperados y proporcionan feedback de inmediato.
- Escribir tests JUnit no supone mucho tiempo.
- Los tests JUnit incrementan la estabilidad del software.
- Los tests JUnit se escriben en Java y son gratis.
- Los tests JUnit son tests del desarrollador.

A propósito de esta última característica, los tests JUnit van a permitir a los desarrolladores optimizar su propio trabajo. Al contrario que los tests funcionales, que tratan al sistema como una caja negra y aseguran que el software funciona como una totalidad, las unidades de tests siguen una visión de sistema como caja blanca (todo ello explicado en puntos anteriores de la unidad), por lo que están escritos para probar los bloques de construcción básicos del sistema desde dentro.

Autoevaluación

Pregunta

¿Qué tipo de pruebas estás asociadas a los tests unitarios?

Respuestas

Pruebas de caja blanca

Pruebas de caja negra

Retroalimentación

Correcto. Las pruebas de caja blanca se centran en los detalles procedimentales del software, por lo que su diseño está fuertemente ligado al código fuente.

Por otro lado, recuerda que las pruebas de caja negra analizan la compatibilidad entre las interfaces de cada uno de los componentes del software, y sin tener en consideración la lógica interna del sistema.

Incorrecto. Las pruebas de caja negra se centran en las funcionalidades del software en su conjunto, analizando la compatibilidad entre las interfaces de cada uno de los componentes del software y sin tener en consideración la lógica interna del sistema.

Solución

1. Opción correcta
2. Incorrecto

2.8.2.1.- Diseño de tests JUnit

JUnit está integrado en los principales Entornos de Desarrollo Integrado (IDE).

A la hora de desarrollar un test de JUnit, se deben llevar a cabo los siguientes pasos:

1-Escribir un test

Un test en JUnit tiene como fin probar un único componente de software.

Para escribir un test, se deben seguir los pasos siguientes:

1. Definir una subclase de TestCase.
2. Sobrescribir el método setUp() para inicializar el objeto a probar.
3. Sobrescribir el método tearDown() para liberar el objeto a probar.
4. Definir uno o más métodos testXXX() públicos que prueben el objeto y aseveren los resultados esperados.
5. Definir un método factoría suite() estático que cree un TestSuite que contenga todos los métodos testXXX() del TestCase.
6. Opcionalmente, definir un método main() que ejecute el TestCase por lotes.

Los métodos descritos anteriormente van acompañadas de una serie de anotaciones.

2-Anotaciones

Las anotaciones son extremadamente importantes al crear JUnits, ya que le va a permitir identificar las partes fundamentales de la prueba unitaria. Todas ellas empiezan por el carácter '@', y cada una tiene una finalidad:

- @Before identifica un método que debe ejecutarse antes de cada método de prueba en la clase. Por lo general, se usa para actualizar o restablecer el estado necesario para que los métodos de prueba se ejecuten correctamente.
- @After identifica un método que debe ejecutarse después de cada método de prueba en la clase. Se puede utilizar para restablecer variables, eliminar archivos temporales, etc.
- @Ignore especifica que no se debe ejecutar un método de prueba.
- @BeforeClass identifica un método que debe ejecutarse una vez antes de ejecutar cualquier método de prueba.
- @AfterClass identifica un método que debe ejecutarse una vez después de ejecutar todos los métodos de prueba.

Caso práctico

Para ilustrar mejor la creación de JUnits, nada mejor que mostrar un ejemplo. La siguiente prueba JUnit va a verificar el correcto funcionamiento de un método simple (parte izquierda) que convierte la temperatura de grados centígrados (ºC) a grados Fahrenheit (ºF). Para ello, se programa una prueba JUnit (parte derecha) creada para probar dicho método *convertirAFahrenheit*.

```java
public class Conversor {

    public double convertirAFahrenheit (double gradosC) {
        double gradosF = gradosC * 9/5 + 32;
        return gradosF;
    }
}
```

```java
package conversortemperatura;

import org.junit.After;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;
import static org.junit.Assert.*;

public class ConversorTest {

    public ConversorTest() {
    }

    @BeforeClass
    public static void setUpClass() {
    }

    @AfterClass
    public static void tearDownClass() {
    }

    @Before
    public void setUp() {
    }

    @After
    public void tearDown() {
    }

    /**
     * Test of convertirAFahrenheit method, of class Conversor.
     */
    @Test
    public void testConvertirAFahrenheit() {
        System.out.println(x: "convertirAFahrenheit");
        double gradosC = 0.0;
        Conversor instance = new Conversor();
        double expResult = 0.0;
        double result = instance.convertirAFahrenheit(gradosC);
        assertEquals(expected: expResult, actual: result, delta: 0);
        // TODO review the generated test code and remove the default call
        fail(message: "The test case is a prototype.");
    }
}
```

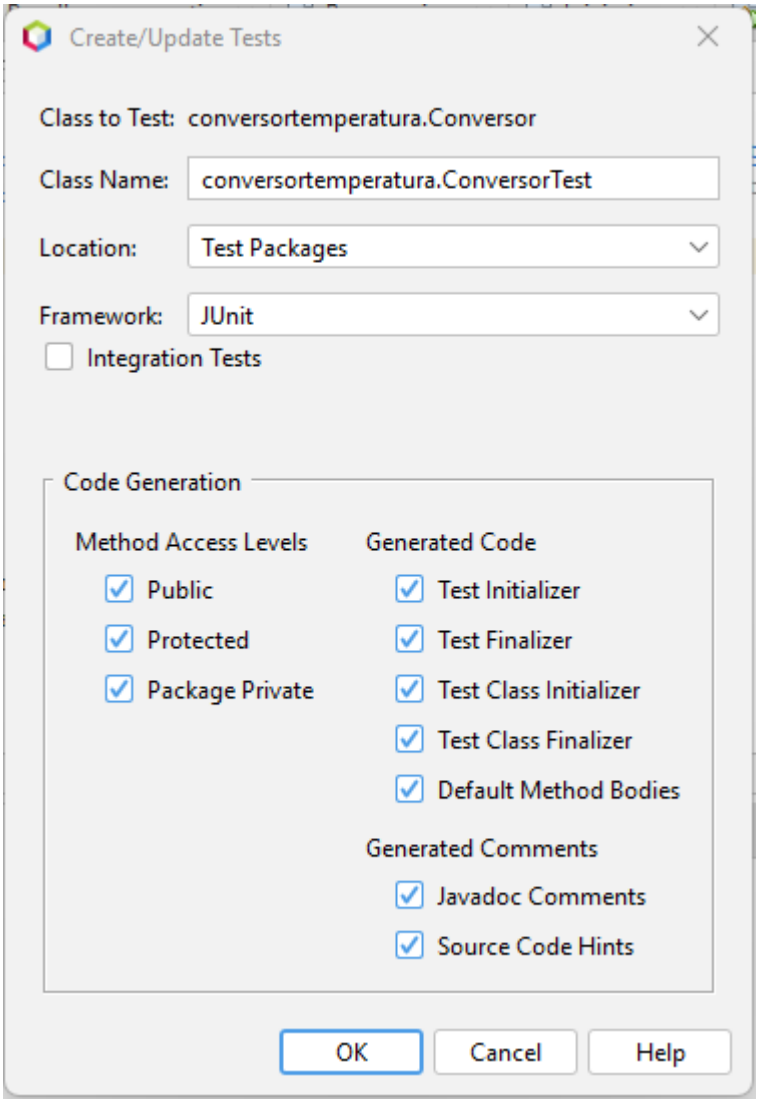
Tomás Montero. *Ejemplo clase JUnit* (CC BY)

2.8.2.2.- Crear un test JUnit en Netbeans

Pasos para obtener la estructura de un test JUnit en el IDE NetBeans

Para obtener la estructura básica de una clase JUnit que permita probar el funcionamiento aislado de una clase desarrollada, basta con:

1. pulsar botón derecho sobre la clase a probar.
2. seleccionar la opción Tools -> Create/Update Tests.
3. Tras esto, se abre una ventana de diálogo como la que se muestra a continuación:



Tomás Montero. *Creación de test JUnit en NetBeans* (CC BY)

Como se puede observar, se indica la clase a probar, el nombre de la clase de test (debe terminar con el sufijo Test), el paquete donde se va a ubicar, y es posible seleccionar qué métodos se van a probar, entre públicos, protegidos y privados; y también se ofrece la opción de incluir código basado en las anotaciones y métodos a nivel de prueba explicados en el apartado anterior (setUpClass, tearDownClass, setUp y tearDown).

Una vez generada la clase, a nivel básico será necesario editar el contenido de los métodos con etiqueta @Test. Para el ejemplo anterior, se ha generado el siguiente método:

```
@Test
public void testConvertirAFahrenheit() {
    System.out.println(x: "convertirAFahrenheit");
    double gradosC = 0.0;
    Conversor instance = new Conversor();
    double expResult = 0.0;
    double result = instance.convertirAFahrenheit(gradosC);
    assertEquals(expected: expResult, actual: result, delta: 0);
    // TODO review the generated test code and remove the default call
    // to fail.
    fail(message:"The test case is a prototype.");
}
```

Tomás Montero. *Test autogenerado* (CC BY)

Como lo que se pretende es probar el corrector funcionamiento de un método que convierte de grados Celsius a grados Fahrenheit, será necesario modificar los datos de entrada, y después verificar que la salida es la correcta (coincide con el resultado esperado), es decir, que el valor de la variable *result* coincide con el de *expResult.*

```
@Test
public void testConvertirAFahrenheit() {
    System.out.println(x: "convertirAFahrenheit");
    double gradosC = 24.0;
    Conversor instance = new Conversor();
    double expResult = 75.2;
    double result = instance.convertirAFahrenheit(gradosC);
    assertEquals(expected: expResult, actual: result, delta: 0);
}
```

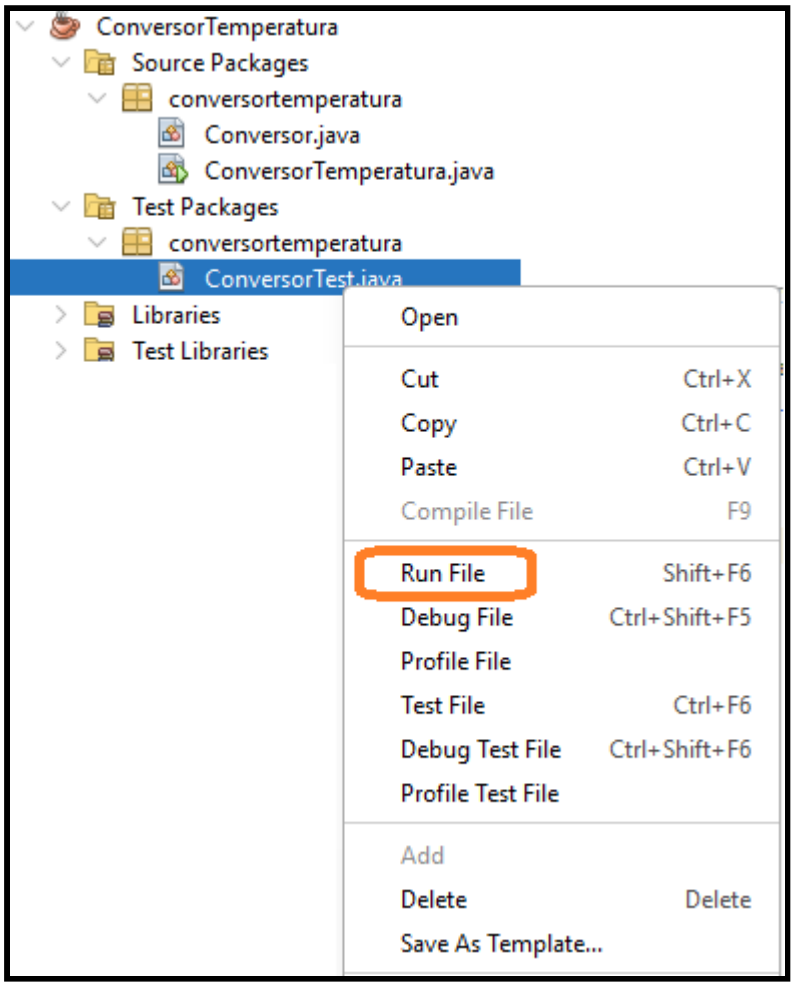
Tomás Montero. *Método JUnit editado* (CC BY)

Para ello se emplea el método assertEquals, que se encarga de aseverar que los valores de expResult y result son iguales, en cuyo caso el test finalizará bien, o de manera errónea si sus valores son distintos.

2.8.2.3.- Ejecución de tests JUnit

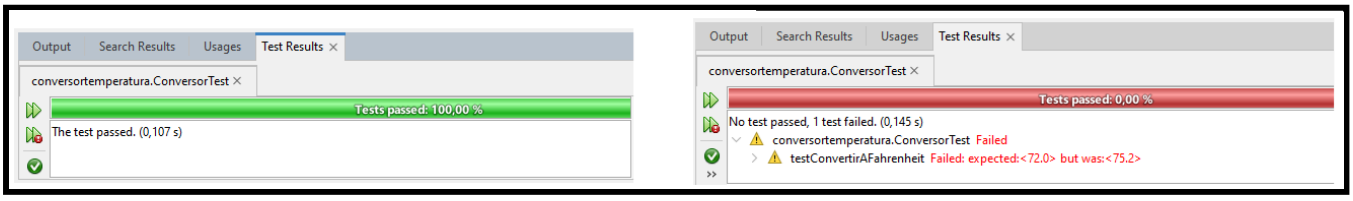
Ejecución de un test JUnit en NetBeans

La ejecución de una clase de test JUnit en NetBeans puede hacerse fácilmente seleccionando la clase en cuestión, con el botón derecho, y seleccionar Run File:



Tomás Montero. *Ejecución de un test* (CC BY)

El resultado de la ejecución podrá ser exitoso (imagen de la izquierda), cuando la conversión del método que estamos probando se realice correctamente; o fallido (imagen de la derecha), cuando el resultado devuelvo por el método a probar no coincida con el valor esperado:



Tomás Montero. *Resultado test ok y ko* (CC BY)

2.8.3.- Framework Mockito

Introducción

Mockito es un framework de código abierto para crear pruebas unitarias en Java. Permite la creación de objetos dobles de prueba, es decir, objetos simulados, con el objetivo fundamental de aislar la clase de prueba.

Los objetos Mock nos permiten simular ser un objeto real y eliminan dependencias, permitiendo que los test se ejecuten de forma aislada.

Se basa en el principio de un objeto *mock* que simula el comportamiento de otro objeto. La implementación de test en Mockito se hace en varias etapas:

1-Creación del objeto mock.

2-Descripción del comportamiento esperado.

3-Uso de los mocks.

4-Verificación de que la interacción con los mocks sea correcta.

¿Cuándo y por qué utilizar frameworks como Mockito?

Cuando existen clases Java que se desean probar con JUnit pero, a su vez, tienen dependencias de otras clases muy complejas, esta circunstancia puede obligarnos a montar toda una infraestructura para poder construir y que funcione la clase que queremos testear.

> Aquí es donde frameworks como Mockito pueden ser muy útiles, ya que son capaces de simular esas clases complejas que necesita la clase que realmente se desea probar.

Un ejemplo de esta problemática se puede presentar a la hora de probar una clase que lee un valor de una base de datos, se conecta luego a un servidor para obtener otro valor y termina mostrando un mensaje con los valores concatenados por pantalla.

Hacer un test de Junit de esto, de la forma "tradicional", sería complejo, porque se debería tener levantada una base de datos con una cadena específica que esperemos en el test y una conexión a un servidor para recuperar un valor. Sin embargo, gracias a Mockito, tanto la conexión con la base de datos como al servidor se podría simular, de manera que el test JUnit se realizaría de forma unitaria sobre la clase que tiene como funcionalidad principal concatenar dichos valores.

2.8.3.1.- Situación de partida donde usar mockito

¿Cuándo va a ser necesario mockear?

A la hora de probar de manera unitaria una clase, va a ser necesario simular el comportamiento de las otras clases de las que dependa.

A continuación se va a explicar un ejemplo sencillo, perteneciente a una empresa de cambio de divisas, formado únicamente por dos clases:

- Tasa, que se va a encargar de calcular el importe total que le va a cobrar la empresa a un usuario por cambiar el dinero de dólares a euros (la empresa se lleva un 10% del importe cambiado).
- Conversor, que obtiene el valor actual de conversión de dólares a euros (el cual puede cambiar de un día para otro), recibe los dólares a convertir y devuelve los euros equivalente.

El código de las clases sería el siguiente, y es importante observar como la clase Tasa va a tener un objeto del tipo Conversor para poder convertir de dólares a euros:

```java
package divisas;

public class Tasa {
    private Conversor conversor;

    public double calcularImporteConversion (double dolares) {
        conversor = new Conversor();
        double euros = conversor.convertirAEuros(dolares);
        double importeTotal = euros * 1.1;

        return importeTotal;
    }
}
```

```java
package divisas;

public class Conversor {

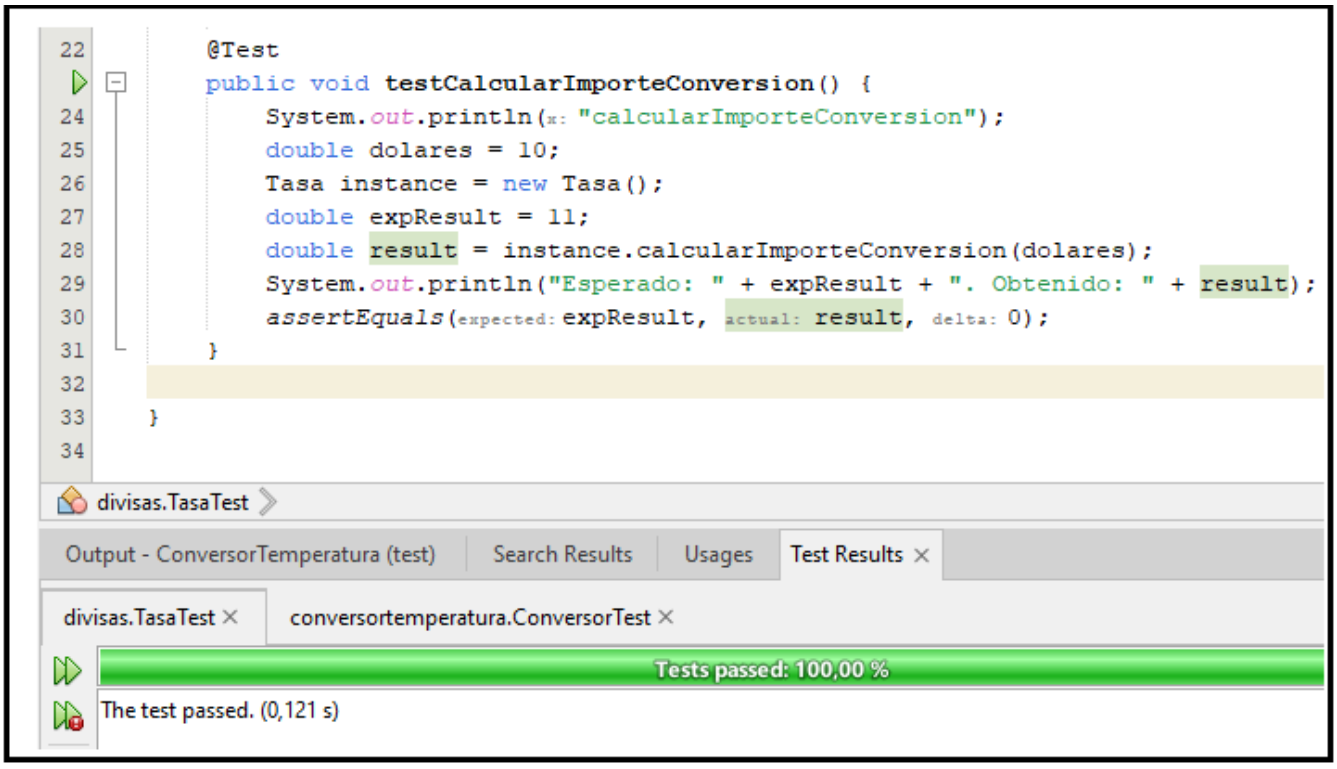
    private double FACTOR_CONVERSION_DOLAR_A_EUROS = 1;

    public double convertirAEuros(double dolares) {
        return dolares * FACTOR_CONVERSION_DOLAR_A_EUROS;
    }

}
```

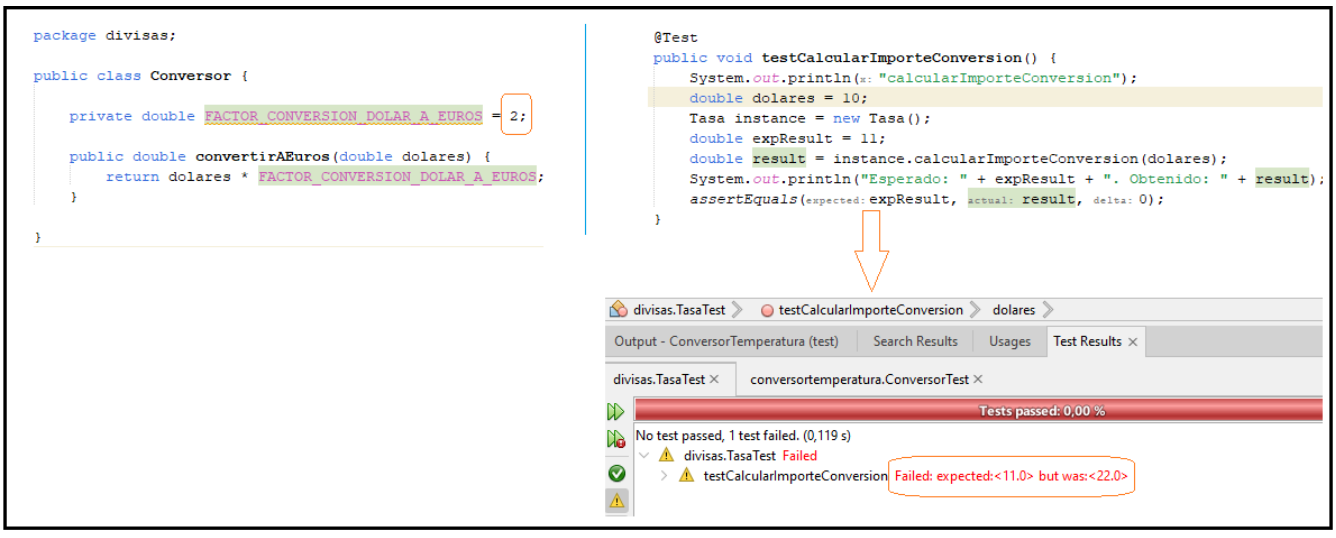
Tomás Montero. *Clases Tasa y Conversor* (CC BY)

Suponiendo que se van a convertir de 10 dólares a euros, que el cambio está a 1 dolar = 1 euro y que la empresa que se queda con un 10% del dinero convertido, el importe total de la conversión será de 11 euros. Por tanto, el test JUnit que permita probar la funcionalidad del método *Tasa.calcularImporteConversion* será el siguiente:



Tomás Montero. *JUnit ok* (CC BY)

Sin embargo, ¿qué sucederá si la conversión entre dólares y euros varía de un día para otro y un dólar pasa a valer, en lugar de 1 euros, 2 euros? Pues que el test anterior fallará, porque realmente no es un test unitario, ya que el método *Tasa.calcularImporteConversion* depende de la funcionalidad del método Conversor.*convertirAEuros* (y del resultado que éste devuelva).



Tomás Montero. *Error JUnit* (CC BY)

2.8.3.2.- Crear y ejecutar un test con Mockito

Test unitario con Mockito

El primer paso para la solución consiste en NO realizar ningún *new* sobre clases externas a la que se va a probar. También se debe simular toda llamada a cualquier método, sea dentro de la clase a probar y de otra clase. Esto es importante a la hora de hacer test unitarios, ya que se debe simular cualquier funcionalidad que no pertenezca a la del test de JUnit.

Para el ejemplo del apartado anterior, donde Tasa.calcularImporteConversion llama internamente al método de otra clase, Conversor.convertirAEuros, se debe *mockear* (simular) esta llamada, de manera que nos aseguramos que siempre se va a devolver el valor que se está esperando. Así, se aislará el JUnit del método que se está probando, que es Tasa.calcularImporteConversion.

Una vez se importe la biblioteca de **Mockito** en el proyecto (del tipo mockito-all-1.10.19.jar), el test JUnit quedará de la siguiente forma:

```
package divisas;

import org.junit.Test;
import static org.junit.Assert.*;
import org.mockito.Mockito;

public class TasaTest {

    public TasaTest() {
    }

    @Test
    public void testCalcularImporteConversion() {
        System.out.println(x: "calcularImporteConversion");
        double dolares = 10;
        Tasa instance = new Tasa();
        double expResult = 11;

        Conversor conversor = Mockito.mock(classToMock:Conversor.class);
        Mockito.when(methodCall: conversor.convertirAEuros(dolares)).thenReturn(value: 10.0);

        double result = instance.calcularImporteConversion(dolares);

        System.out.println("Esperado: " + expResult + ". Obtenido: " + result);
        assertEquals(expected: expResult, actual: result, delta: 0);
    }
}
```

Tomás Montero. *JUnit mockeado* (CC BY)

Como se puede ver en la imagen anterior, cuando dentro del método a probar se invoque a *conversor.convertirAEuros*, entonces se devolverá un valor fijo (el valor esperado) y, por tanto, el test se centrará en ver si se realiza correctamente la funcionalidad del método *calcularImporteConversion*, es decir, si se aplica el porcentaje.

Para saber más

Otro ejemplo de JUnit empleando Mockito: https://chuidiang.org/index.php?title=Ejemplo_sencillo_con_Mockito

2.9.- De usuario.

Caso práctico

Parece que vamos terminando las pruebas de la aplicación. ¿Cómo la verá el cliente? ¿Le resultará amigable y fácil de utilizar? Siempre perseguimos la satisfacción del usuario, por lo tanto, su opinión en cuanto a la apariencia de nuestro proyecto es algo que cada vez cobra más importancia.

"Ha llegado la hora de probar la usabilidad de la aplicación", anuncia **Ada**.

Los usuarios finales van a ser los que utilicen nuestra aplicación y, por tanto, ésta debe satisfacer sus necesidades, objetivos y expectativas.

La prueba de usuario (también conocida como de usabilidad) consiste en asegurar que la interfaz de usuario (GUI) de la aplicación sea amigable, intuitiva y que funcione correctamente. Se determina la calidad del software en la forma en la que el usuario interactúa con el sistema, considerando la facilidad de uso y satisfacción del cliente.

Se define la usabilidad de una aplicación como un parámetro que mide de qué forma los usuarios la utilizan: cómo navegan a través de sus pestañas, ventanas, buscadores, etc. Los análisis de usabilidad mostrarán lo que los usuarios realmente van a ver.

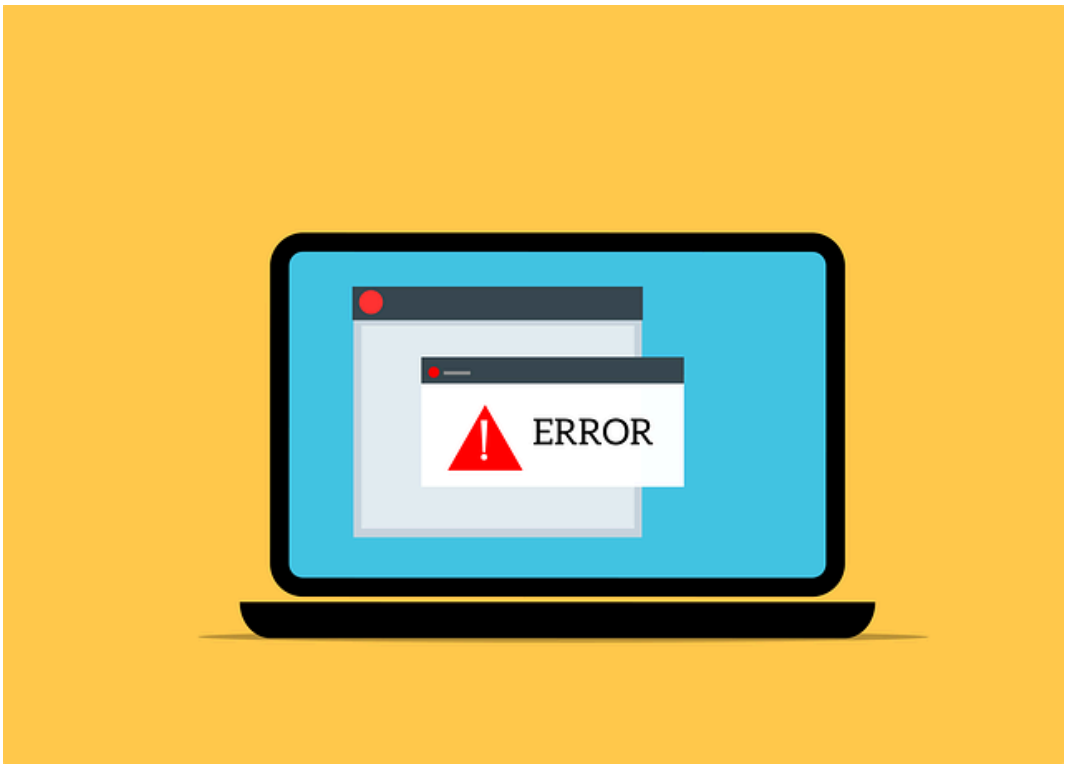
Podemos evaluar la usabilidad de una aplicación midiendo la manera en que el usuario realiza una tarea concreta, el tiempo y número de clics que supone acabarla y los errores que comete durante el proceso.

Pixabay.com. *Error* (Licencia Pixabay)

Es obvio que el estudio de la reacción de los individuos ante nuestra aplicación es un proceso complicado, pero existen varias métricas que implican un grado de fiabilidad que queda cubierto en un entorno de pruebas automatizadas.

Para verificar la facilidad de uso de una aplicación se pueden incluir en los ensayos de pruebas:

- Encuestas y entrevistas.
- Grupos de enfoque.



Pruebas con herramientas de software avanzado.

Evaluaciones realizadas por expertos.

Como en otras muchas ocasiones, cuanto antes se realicen las pruebas de usabilidad, mucho mejor: no sólo resultará más barato a largo plazo, sino que se adaptará también mejor al programa (si se corrigen los problemas a tiempo no habrá que construir sobre errores).

Autoevaluación

Pregunta

Hemos casi terminado de probar la aplicación. Te encanta ver el resultado de tu trabajo y piensas en lo bien que ha quedado. Observas que cuando intentamos alguna opción no válida, se abre una ventana que dice "la acción no ha podido ser ejecutada con éxito". Sea cual sea la acción no válida, el mensaje siempre es el mismo. Si nuestro cliente es una cadena hotelera y la persona que va a usar la aplicación no tiene grandes conocimientos de informática, ¿crees que el mensaje debería ser diferente?

Respuestas

No, está suficientemente claro.

Sí, debería concretar el error para que el usuario sepa dónde se ha equivocado.

Retroalimentación

No es correcto, el parámetro de usabilidad de la aplicación no satisface los requerimientos.

Efectivamente, por las características del cliente, para que no se sienta “perdido”.

Solución

1. Incorrecto
2. Opción correcta

2.10.- De aceptación.

Caso práctico

Una vez todo terminado, el producto está ya listo para la entrega. En BK hay bastantes nervios. Ha sido un proyecto complejo y esperan ansiosamente la respuesta del cliente.

"Nos disponemos a validar la aplicación", anuncia María. "Que sea lo que el cliente requirió en su momento es el último paso para finalizar.



Pixabay.com. *Valoración positiva* (Licencia Pixabay)

La prueba de aceptación (o de validación) sirve para comprobar que el software cumple con los requerimientos iniciales y que funciona de acuerdo con lo que el cliente esperaba de él, es decir, satisfaciendo sus expectativas. Por eso, y para que la prueba sea objetiva, es el mismo cliente quien la realiza. Podemos escuchar comentarios como: "esto no era lo que yo quería" o "esto no es lo que yo necesito", e incluso "esto no es lo que yo pedí".

En esta prueba se evalúa el grado de calidad del software en relación a su uso. Es definida por el usuario cliente y preparada por el equipo de desarrollo, aunque la ejecución y aprobación final corresponden al usuario.

La validación del sistema se consigue mediante la realización de pruebas de caja negra que demuestran la conformidad con los requisitos.

Para su ejecución se suelen emplear dos técnicas de aceptación: alfa y beta.

Pruebas alfa

Las lleva a cabo el cliente en el lugar donde se ha desarrollado la aplicación y en presencia del desarrollador. El cliente utilizará la aplicación como lo haría en su lugar de trabajo. Se utiliza un entorno controlado con las mismas condiciones que hay en el lugar de trabajo habitual. Una vez realizadas, se documentan los resultados.

Pruebas beta

Las lleva a cabo el cliente, pero ya en su lugar de trabajo. A diferencia de las pruebas alfa, el desarrollador ya no está presente durante su realización, por lo que estas pruebas se denominan "en vivo", puesto que no se ejecutan en un entorno controlado. El cliente registrará todos los problemas que encuentre durante la prueba beta e irá informando al desarrollador a intervalos regulares de tiempo. Éste realizará las correcciones y modificaciones pertinentes y preparará una nueva versión del producto.



Ministerio de Educación y Formación Profesional (Elaboración propia)

En la prueba de aceptación se evalúa:

- **Que se satisfacen los requisitos funcionales.**
- **Que se alcanzan los requisitos de rendimiento.**
- **Que la documentación es correcta e inteligible.**

Lista desordenada de pruebas

Ordena los siguientes tipos de pruebas, según deben realizarse tras finalizar la fase de implementación, y siguiendo el modelo en V (más información, aquí):

- Pruebas unitarias
- Pruebas de integración
- Pruebas de regresión
- Pruebas funcionales
- Pruebas de sistema
- Pruebas de aceptación